Green University of Bangladesh

Department of Computer Science and Engineering (CSE)
Semester: (Summer, Year: 2026), B.Sc. in CSE (Day)

Intelligent Sudoku Solver and Generator using Search Algorithms

---

PROJECT REPORT

Course Title: Artificial Intelligence Lab
Course Code: CSE-316
Section: 242_D3

Students Details

Name	ID
Md. Reahoon Zannah	223002038
Md. Aminul Islam	223002004

Submission Date: 23/08/2026
Course Teacher's Name: Sian Ashsad

[For teachers use only: Don't write anything inside this box]

Lab Project Status	
Marks:	**Signature:**
Comments:	**Date:**

Contents

Chapter 1

Introduction

1.1 Overview

Sudoku is a logic-based number placement puzzle. The objective is to fill a 9×9 grid with digits so that each column, each row, and each of the nine 3×3 sub grids contain all of the digits from 1 to 9. From a computer science perspective, Sudoku is a classic Constraint Satisfaction Problem (CSP). This project focuses on building an automated Sudoku solver and generator using Python. It visualizes how different artificial intelligence algorithms, ranging from basic blind search to advanced heuristics, navigate the problem space to find a solution. [1]

1.2 Motivation

Understanding how search algorithms work can be difficult when looking only at command line outputs. We wanted to build a visual tool that shows exactly how a computer "thinks" when solving a complex puzzle. By watching algorithms backtrack, evaluate cells, and apply constraints in real-time, students and developers can easily grasp the differences in algorithm efficiency.

1.3 Problem Definition

1.3.1 Problem Statement

The main challenge is designing a system that can not only solve any valid Sudoku puzzle but also visualize the process without freezing the user interface. Basic algorithms like Breadth-First Search struggle with the massive state space of Sudoku, leading to memory issues or excessive solve times. We need to implement and compare smarter algorithms like Minimum Remaining Values (MRV) and Forward Checking to handle the strict constraints of the 9×9 grid efficiently.

1.3.2 Complex Engineering Problem

Table 1.1: Attributes of Complex Engineering Problems and Their Explanations

Name of the Attributes	**Explain how to address**
P1: Depth of knowledge required	Requires a solid understanding of graph theory, search algorithms, constraint satisfaction, and heuristic optimization.
P2: Range of conflicting requirements	Must balance the heavy computational load of search algorithms with a smooth, responsive graphical user interface.
P3: Depth of analysis required	Involves tracking algorithm efficiency through step counts and cell evaluations to compare performance objectively.
P4: Familiarity with modern tools and techniques	Requires proficiency in Python, the Tkinter GUI framework, and multi-threading concepts.
P5: Extent of applicable codes	Integration of modular object-oriented programming, separating the UI logic from the mathematical solver logic.

1.4 Objectives

- To implement a functional, interactive Sudoku grid using Python and Tkinter.
- To code and compare five different search algorithms: DFS, BFS, Heuristic Search, MRV, and Forward Checking.
- To use multi-threading so the graphical interface remains responsive while the algorithms calculate in the background.
- To create a puzzle generator that creates solvable boards at four difficulty levels (Easy, Medium, Hard, and Very Hard).
- To track and display algorithm performance metrics such as total steps and cell evaluations.

1.5 Application

1. Educational tool for teaching Artificial Intelligence search strategies.
2. Logic puzzle game for general users.
3. Automated testing ground for constraint satisfaction theories.
4. Foundation for solving other real-world scheduling and allocation problems.

Chapter 2

Design/Development/Implementation of the Project

2.1 Introduction

The project is built entirely in Python. We chose Tkinter for the graphical interface because it is lightweight and built into standard Python distributions. The architecture is split into three main classes: SudokuSolver for the math and algorithms, SudokuGenerator for building new puzzles, and SudokuApp combined with SudokuGrid to handle what the user sees and interacts with.

2.2 Project Details

The system features several distinct functional modules:

- **Interactive Grid:** A 9×9 Sudoku board where users can manually enter numbers or observe the AI solving the puzzle in real time.
- **Control Panel:** Provides buttons to select different solving algorithms, generate new Sudoku puzzles, and manage the visualization process using *Solve*, *Pause*, *Resume*, and *Stop* controls.
- **Speed Controller:** Includes a slider that allows users to adjust the speed of the solving visualization according to their preference. [2]
- **Statistics Tracker:** Displays real-time information about the solving process, including the current status, elapsed time, number of steps taken, and total cell evaluations performed.

2.3 Implementation

Workflow

1. The user launches the application, which generates a blank 9×9 Sudoku grid.
2. The user either enters the puzzle manually or selects a difficulty level to generate a new Sudoku puzzle.
3. The user chooses a solving algorithm from the right-hand menu. [3]
4. Upon clicking the *Solve* button, the system starts a background thread to execute the selected algorithm while continuously updating the GUI with current cell evaluations and final number placements.

Tools and Environment

- **Programming Language:** Python 3 is used as the primary programming language for implementing the Sudoku solver and visualization system.
- **GUI Framework:** Tkinter (`ttk`) is utilized to design and manage the graphical user interface.
- **Standard Libraries Used:**
  - **time:** Measures execution time and controls visualization delays.
  - **random:** Generates random Sudoku puzzles and selects random values.
  - **queue:** Facilitates communication between the solving thread and the GUI.
  - **copy:** Creates deep copies of Sudoku boards to preserve the original state.
  - **enum:** Defines enumerations for solver states and algorithm selection.
  - **threading:** Executes solving algorithms in a background thread to keep the GUI responsive.

Implementation Details

In the UI is divided cleanly for Sudoku grid. The left side holds the custom SudokuGrid widget, styled with distinct 3×3 box borders. The right side contains modern styled buttons for algorithm selection and puzzle generation.

A critical implementation detail was using the threading library. If we ran a while loop for the DFS algorithm directly on the main Tkinter thread, the window would freeze until the puzzle was solved. By placing the solver in solve-thread(), the UI stays awake, allowing the user to click the "Pause" or "Stop" buttons at any time.

Code Repository

1. Github repository: [Sudoku Game Solver]

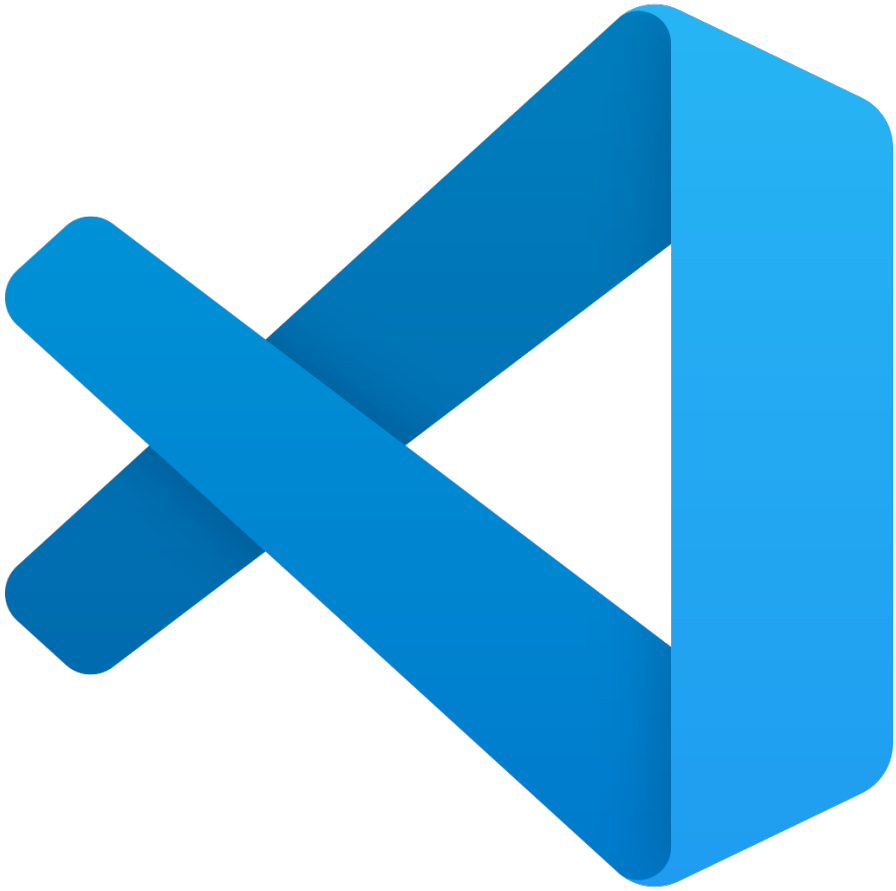

Figure 2.1: Visual Studio Code (compiler)

2.4 Algorithms

- Algorithm of my project. [4]

Algorithm 1 Depth-First Search (DFS)

1: Input: Sudoku Board
2: Output: Solved Sudoku Board
3: Find the first empty cell.
4: Try numbers 1 through 9 sequentially.
5: If a number satisfies all Sudoku constraints, place it.
6: Recursively continue with the next empty cell.
7: If no valid number exists, backtrack and try another value.
8: Repeat until the board is solved.

Algorithm 2 Breadth-First Search (BFS)

Input: Sudoku Board
Output: Solved Sudoku Board or Failure
Initialize a queue with the initial board state.
While the queue is not empty:
    Remove the front board state.
    Generate all possible valid successor states.
    Insert successors into the queue.
Stop if the solution is found or 10,000 states are explored.
Return the solved board or terminate.

Algorithm 3 Heuristic Search

Input: Sudoku Board
Output: Solved Sudoku Board
Count the frequency of each number already on the board.
Select the first empty cell.
Try candidate numbers from least frequent to most frequent.
If a valid placement is found, continue recursively.
Otherwise, backtrack and test another candidate.
Return the solved board.

Algorithm 4 Minimum Remaining Values (MRV)

Input: Sudoku Board
Output: Solved Sudoku Board
For each empty cell, determine all legal values.
Select the cell with the fewest legal values.
If no legal value exists, backtrack immediately.
Place each possible value recursively.
Continue until the puzzle is solved.
Return the solved board.

Algorithm 5 Forward Checking

Input: Sudoku Board
Output: Solved Sudoku Board
Create a domain of possible values for every empty cell.
Select an empty cell and assign a legal value.
Remove that value from the domains of related cells.
If any domain becomes empty, backtrack.
Continue updating domains after every assignment.
Return the solved Sudoku board.

Listing 2.1: Pseudocode for our MRV Implementation:

```
Function solve_mrv(board):
    Find the empty cell 'C' that has the fewest legal numbers
        left.
    If there are no empty cells, return True (Puzzle Solved).

    Get the list of legal numbers for 'C'.
    If the list is empty, return False (Dead end).

    For each number in the legal list:
        Place number in 'C'
        Update GUI

        If solve_mrv(board) is True:
            return True

        Remove number from 'C' (backtrack)
        Update GUI

    return False
```

Chapter 3

Performance Evaluation

3.1 Simulation Environment/Simulation Procedure

The software was tested on a standard desktop environment running Python 3. The testing procedure involved generating puzzles across all four difficulty levels and running each of the five algorithms to compare their step counts, evaluation counts, and physical time to solve (with the visualization delay set to zero for accurate bench-marking).

3.2 Results Analysis/Testing

3.2.1 Depth-First Search vs Breadth-First Search

DFS proved to be a reliable, albeit sometimes slow, method for solving. It easily handles easy and medium puzzles but can get caught in long backtracking loops on hard puzzles depending on the initial empty cell layout. BFS, as expected, performed poorly. Searching level by level in a 9×9 grid causes the queue to grow exponentially. The system correctly triggered our safety cutoff at 10,000 steps to prevent memory overflow.

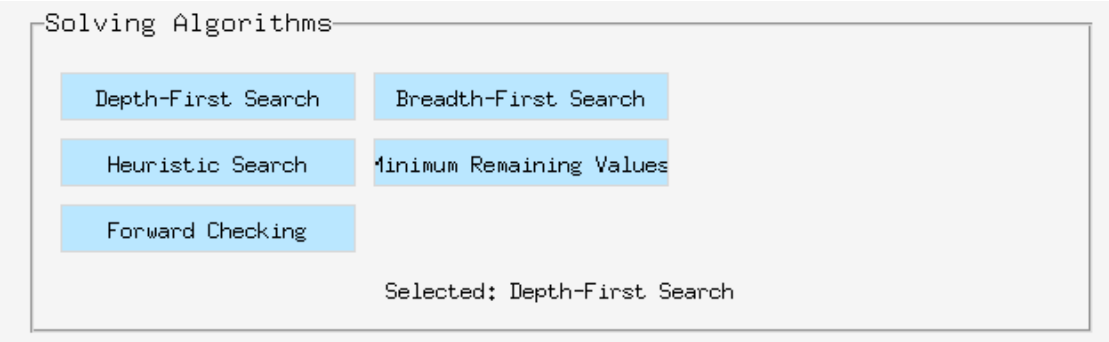

Figure 3.1: The algorithm selection panel allowing users to choose the specific search method to solve the Sudoku board.

3.2.2 Constraint Satisfaction Algorithms

The algorithms utilizing constraint satisfaction concepts completely outperformed blind search.

- The MRV algorithm required significantly fewer steps than DFS. By selecting the most constrained cell first, it minimizes poor decisions early in the search process and reduces unnecessary backtracking.
- Forward Checking proved to be the most efficient at avoiding dead ends. By continuously updating the domains of neighboring cells after each assignment, it eliminates invalid choices in advance and reduces the overall search effort.

3.2.3 Puzzle Generation

The puzzle generator successfully created unique boards. The logic fills the diagonal 3×3 boxes first (which are independent of each other), solves the rest of the board using DFS to ensure a valid complete state, and then removes between 40 to 64 numbers based on the selected difficulty (Easy to Very Hard).

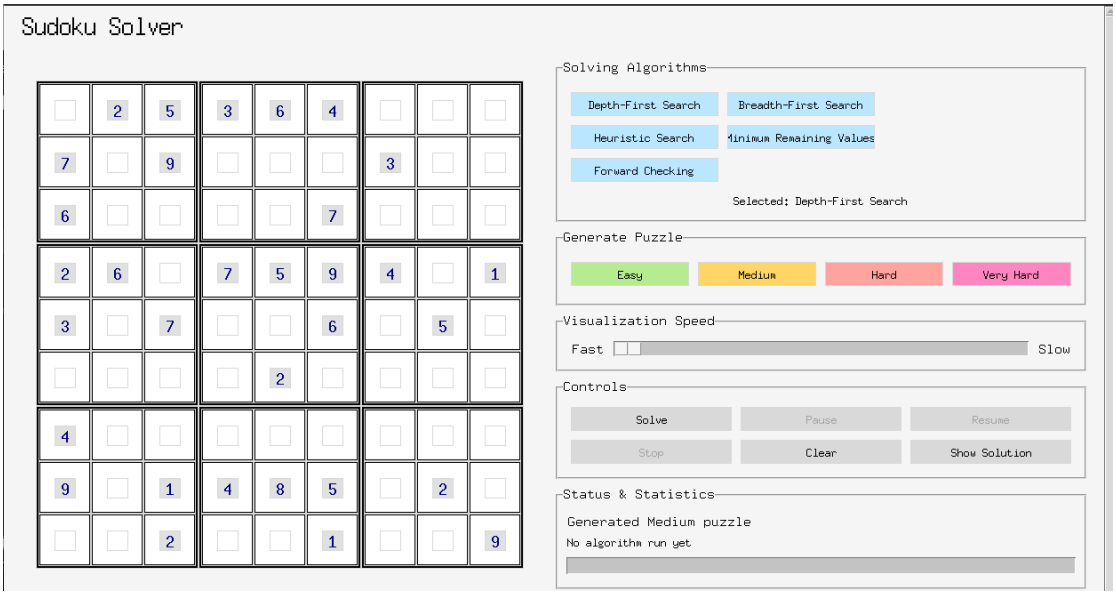

Figure 3.2: Puzzle generation and algorithm visualization.

3.3 Results Overall Discussion

The results clearly illustrate why AI utilizes heuristics and constraints rather than brute force. While a computer is fast enough that DFS can solve a Sudoku board in a fraction of a second without visual delays, the step count difference between DFS and MRV highlights how much computational waste is occurring. The visualizer proved highly

1	2	5	3	6	4	7	9	8
7	4	9	2	1	8	3	6	5
6	8	3	5	9	7	2	1	4
2	6	8	7	5	9	4	3	1
3	1	7	8	4	6	9	5	2
5	9	4	1	2	3	8	7	6
4	5	6	9	3	2	1	8	7
9	7	1	4	8	5	6	2	3
8	3	2	6	7	1	5	4	9

Figure 3.3: Solved sudoku using algorithm

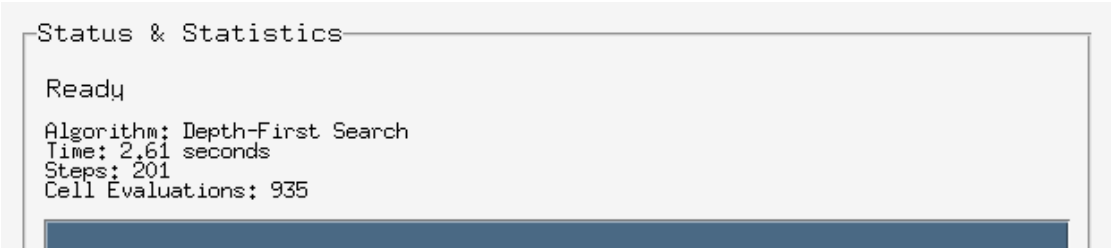

Figure 3.4: The "Status and Statistics" panel displaying the results after running the Depth-First Search algorithm, including total time taken, steps required, and the number of cell evaluations.

effective in showing how Forward Checking trims the search tree compared to the erratic guessing seen in standard DFS.

Chapter 4

Conclusion

4.1 Discussion

This project successfully met all its objectives. We built a clean, user friendly interface that effectively visualizes how different artificial intelligence algorithms solve a Constraint Satisfaction Problem. The integration of multi-threading ensured a stable user experience, and the real-time tracking of steps and cell evaluations provides tangible data for comparing algorithm performance.

4.2 Limitations

- **BFS Inefficiency:** The Breadth-First Search (BFS) algorithm is fundamentally unsuited for solving Sudoku puzzles due to the enormous branching factor. In this application, it serves primarily as a theoretical comparison rather than a practical solving method.
- **Generator Speed:** The puzzle generator employs a simplified approach to verify solution uniqueness. A more robust implementation would require exploring all possible solution paths, which can introduce a slight delay when generating *Very Hard* puzzles.
- **Visual Delay Limits:** Even at the *Fast* visualization setting, Tkinter has inherent rendering limitations that restrict how quickly the interface can redraw the Sudoku grid during intensive backtracking operations.

4.3 Scope of Future Work

- Implement Algorithm X (Dancing Links), which is widely regarded as one of the fastest exact cover algorithms for solving Sudoku puzzles. [5]
- Add image processing capabilities using OpenCV, allowing users to capture a printed Sudoku puzzle with a webcam and automatically import it into the application.

- Introduce a hint system that explains the reasoning behind each move by highlighting logical techniques, such as Naked Pairs or X-Wings, instead of simply filling in the correct number.

Bibliography

[1] C. P. Gomes, B. Selman, and H. Kautz, "Boosting combinatorial search through randomization," *Proceedings of the National Conference on Artificial Intelligence*, pp. 431–437, 1998.

[2] R. M. Haralick and G. L. Elliott, "Increasing tree search efficiency for constraint satisfaction problems," *Artificial Intelligence*, vol. 14, no. 3, pp. 263–313, 1980.

[3] A. K. Mackworth, "Consistency in networks of relations," *Artificial Intelligence*, vol. 8, no. 1, pp. 99–118, 1977.

[4] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

[5] D. E. Knuth, "Dancing links," *Millennial Perspectives in Computer Science*, pp. 187–214, 2000.